

Practical: 1

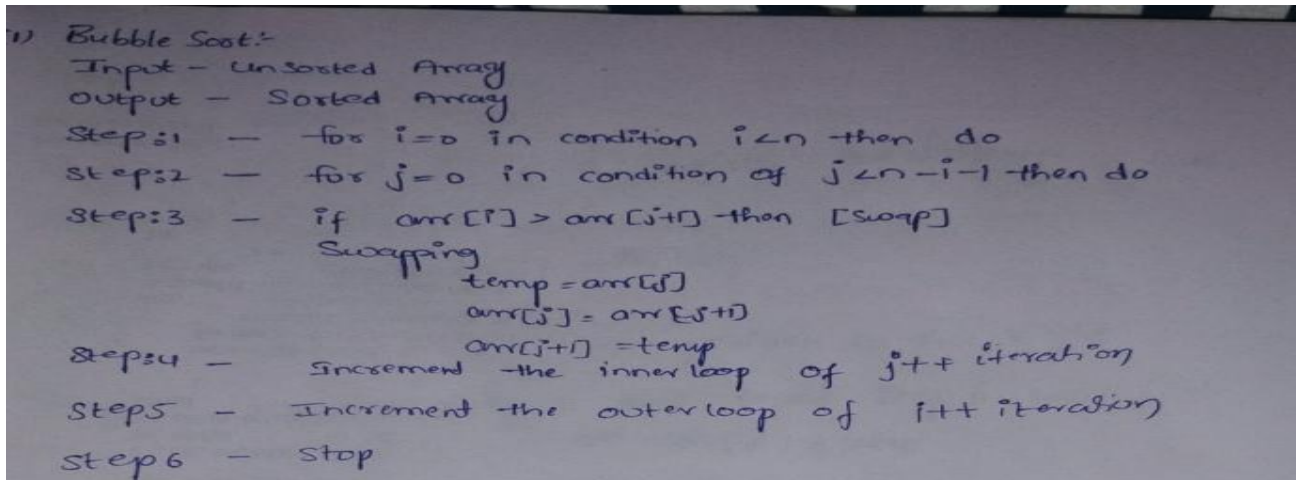
29/07/2026

Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort

Aim: To Implement and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort.

Algorithm:

BUBBLE



Program:

```
#include <iostream>
using namespace std;
```

```
class bubblesort{
public:
    int size;
    int arr[100];
    int i;
    void arrayinput(){
        cout << "enter the size of array" << endl;
        cin >> size;
        cout << "enter the array elements" << endl;
        for (i = 0; i < size; i++){
            cin >> arr[i];
        }
    }

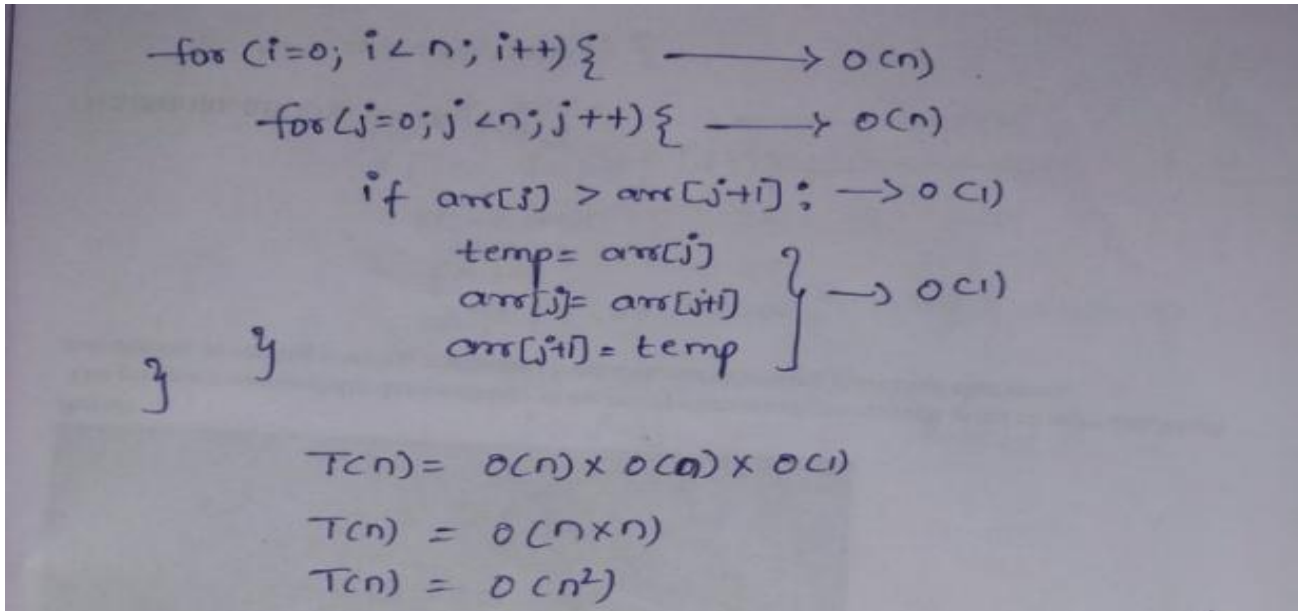
    void printarray(){
        for (i = 0; i < size; i++){
            cout << arr[i] << " ";
        }
        cout << endl;
    }
};
```

```
}  
void bubblelogic(){  
    int i, j, temp;  
    for (i = 0; i < size - 1; i++){  
        for (j = 0; j < size - 1; j++){  
            if (arr[j] > arr[j + 1]){  
                temp = arr[j];  
                arr[j] = arr[j + 1];  
                arr[j + 1] = temp;  
            }  
        }  
    }  
}  
};  
  
int main(){  
    bubblesort bs;  
    bs.arrayinput();  
    cout << "before sorting" << endl;  
    bs.printarray();  
    cout << "after sorting" << endl;  
    bs.bubblelogic();  
    bs.printarray();  
    return 0;  
}
```

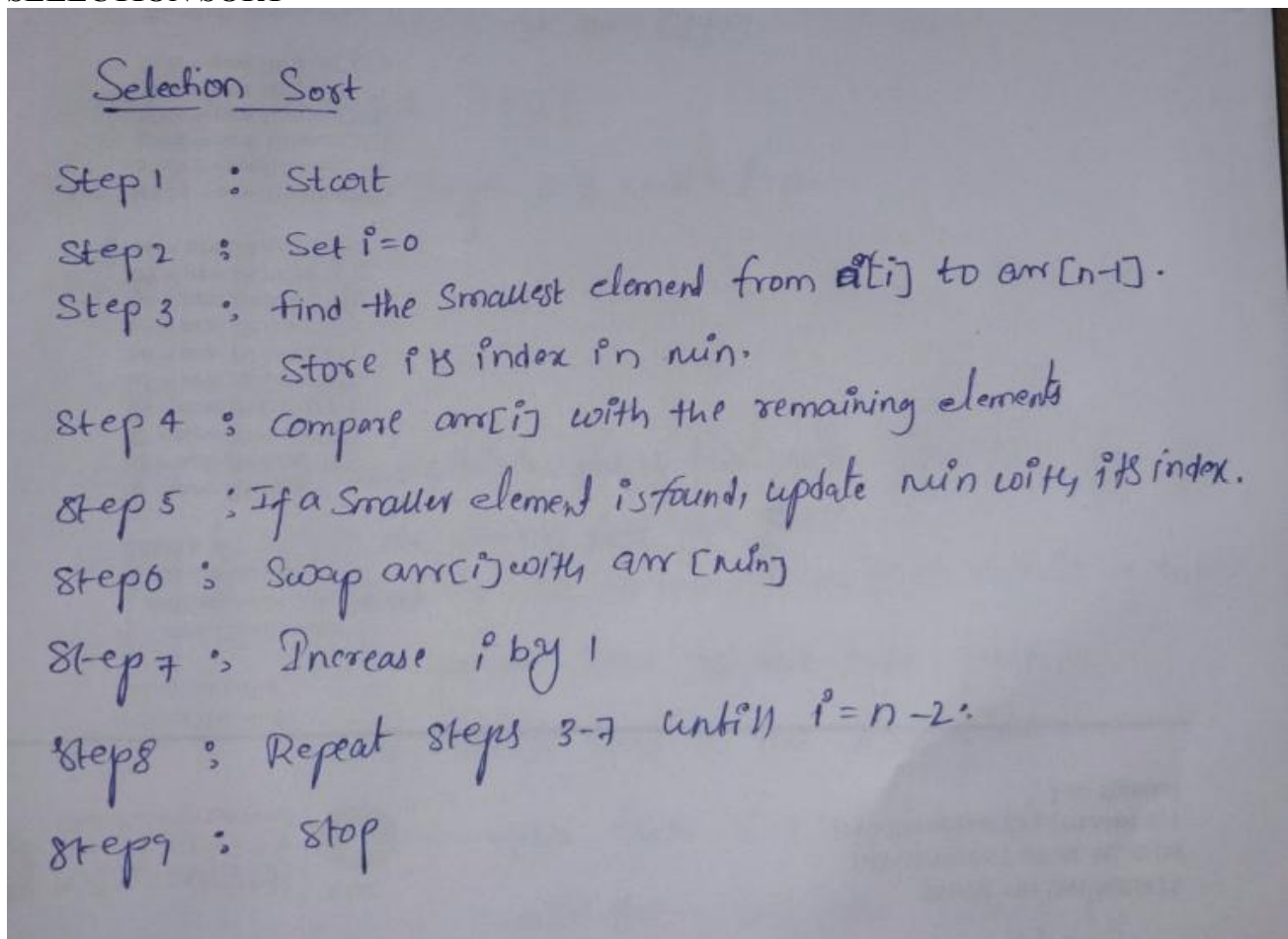
OUTPUT

```
enter the size of array  
5  
enter the array elements  
3  
2  
6  
1  
22  
before sorting  
3 2 6 1 22  
after sorting  
1 2 3 6 22  
  
=== Code Execution Successful ===
```

TIME COMPLEXITY



SELECTION SORT



PROGRAM

```
#include <iostream>
using namespace std;
class selectionsort{
public:
    int size;
    int arr[100];
    int i;
    void arrayinput(){
        cout << "enter the size of array" << endl;
        cin >> size;
        cout << "enter the array elements" << endl;
        for (i = 0; i < size; i++){
            cin >> arr[i];
        }
    }
    void printarray(){
        for (i = 0; i < size; i++){
            cout << arr[i] << " ";
        }
        cout << endl;
    }
    void selectionlogic(){
        int i, j, minIndex, temp;
        for (i = 0; i < size - 1; i++){
            minIndex = i;
            for (j = i + 1; j < size; j++){
                if (arr[j] < arr[minIndex]){
                    minIndex = j;
                }
            }
            temp = arr[i];
            arr[i] = arr[minIndex];
            arr[minIndex] = temp;
        }
    }
};

int main(){
    selectionsort ss;
    ss.arrayinput();
    cout << "before sorting" << endl;
    ss.printarray();
    cout << "after sorting" << endl;
    ss.selectionlogic();
    ss.printarray();
    return 0;
}
```

OUTPUT

```
enter the size of array
5
enter the array elements
2
4
1
66
22
before sorting
2 4 1 66 22
after sorting
1 2 4 22 66

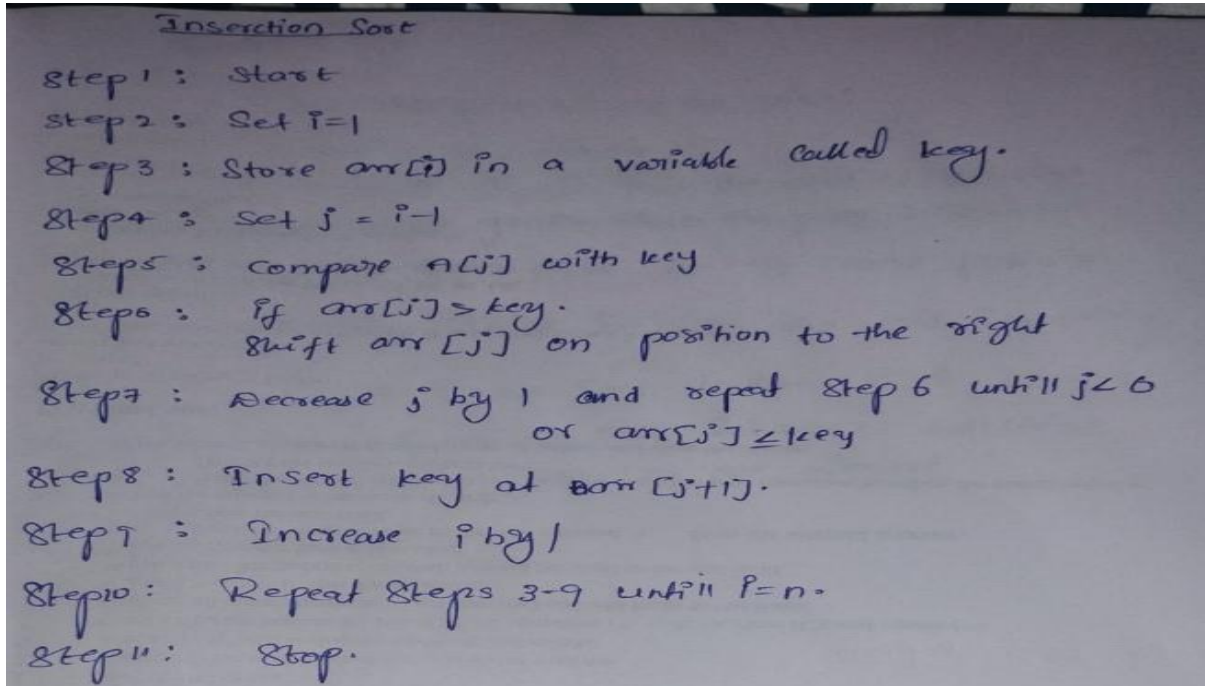
=== Code Execution Successful ===
```

TIME COMPLEXITY

```
void SelectionLogic ( ) {
    int i, j, minIndex, temp;  —————> O(1)
    for (i=0; i < size-1; i++) { —————> O(n)
        minIndex = i;          —————> O(1)
        for (j=i+1; j < size; j++) { —————> O(n)
            if (arr[j] < arr[minIndex]) { —————> O(1)
                minIndex = j; —————> O(1)
            }
        }
        temp = arr[i]; —————> O(1)
        arr[i] = arr[minIndex]; —————> O(1)
        arr[minIndex] = temp; —————> O(1)
    }
}
```

$T(n) = O(1) \times O(n) \times O(1) \times O(n) \times O(1) \times O(1) \times O(1) \times O(1) \times O(1)$
 $= O(n \times n)$
 $= O(n^2)$

INSERTION SORT



PROGRAM

```
#include <iostream>
using namespace std;
class insertionsort{
public:
    int size;
    int arr[100];
    int i;
    void arrayinput(){
        cout << "enter the size of array" << endl;
        cin >> size;
        cout << "enter the array elements" << endl;
        for (i = 0; i < size; i++){
            cin >> arr[i];
        }
    }
    void printarray(){
        for (i = 0; i < size; i++){
            cout << arr[i] << " ";
        }
        cout << endl;
    }
    void insertionlogic(){
        int i, j, key;
        for (i = 1; i < size; i++){
            key = arr[i];
```

```
        j = i - 1;
        while (j >= 0 && arr[j] > key){
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}
};
int main(){
    insertionsort is;
    is.arrayinput();
    cout << "before sorting" << endl;
    is.printarray();
    cout << "after sorting" << endl;
    is.insertionlogic();
    is.printarray();
    return 0;
}
```

OUTPUT

```
enter the size of array
5
enter the array elements
4
3
55
22
6
before sorting
4 3 55 22 6
after sorting
3 4 6 22 55

=== Code Execution Successful ===
```




TIME COMPLEXITY

Insertion

```

void insertionSort() {
    int i, j, key;
    for (i=1; i<size; i++) {
        key = arr[i];
        j = i-1;
        while (j>=0 && arr[j]>key) {
            arr[j+1] = arr[j];
            j = j-1;
        }
        arr[j+1] = key;
    }
}

```

$\rightarrow O(1)$
 $\rightarrow O(n)$
 $\rightarrow O(1)$
 $\rightarrow O(1)$
 $\rightarrow O(n)$
 $\rightarrow O(1)$
 $\rightarrow O(1)$
 $\rightarrow O(1)$

$T(n) = O(1) \times O(n) \times O(1) \times O(1) \times O(n) \times O(1) \times O(1) \times O(1)$
 $= O(n \times n)$
 $= O(n^2)$

MERGE SORT

Step 1 : Start
 Step 2 : If low < high, then find mid = (low+high)/2
 Step 3 : Divide the array into two parts:
 Left part $\rightarrow$ low to mid, Right part $\rightarrow$ mid + 1 to high
 Step 4 : Apply merge sort to the left part
 Step 5 : Apply merge sort to the right part
 Step 6 : Merge the two sorted parts
 Step 7 : Repeat until the complete array is sorted
 Step 8 : Stop

PROGRAM

```
#include <iostream>
using namespace std;
class mergesort{
public:
    int size;
    int arr[100];
    int i;
    void arrayinput(){
        cout << "enter the size of array" << endl;
        cin >> size;
        cout << "enter the array elements" << endl;
        for (i = 0; i < size; i++){
            cin >> arr[i];
        }
    }
    void printarray(){
        for (i = 0; i < size; i++){
            cout << arr[i] << " ";
        }
        cout << endl;
    }
    void merge(int low, int mid, int high){
        int temp[100];
        int i = low;
        int j = mid + 1;
        int k = low;
        while (i <= mid && j <= high){
            if (arr[i] <= arr[j]){
                temp[k] = arr[i];
                i++;
            }
            else{
                temp[k] = arr[j];
                j++;
            }
            k++;
        }
        while (i <= mid){
            temp[k] = arr[i];
            i++;
            k++;
        }
        while (j <= high){
            temp[k] = arr[j];
            j++;
            k++;
        }
    }
};
```

```
    }
    for (i = low; i <= high; i++){
        arr[i] = temp[i];
    }
}

void mergellogic(int low, int high){
    if (low < high){
        int mid = (low + high) / 2;
        mergellogic(low, mid);
        mergellogic(mid + 1, high);
        merge(low, mid, high);
    }
}

};

int main(){
    mergesort ms;
    ms.arrayinput();
    cout << "before sorting" << endl;
    ms.printarray();
    cout << "after sorting" << endl;
    ms.mergellogic(0, ms.size - 1);
    ms.printarray();
    return 0;
}
```

OUTPUT

```
enter the size of array
4
enter the array elements
2
1
4
3
before sorting
2 1 4 3
after sorting
1 2 3 4

=== Code Execution Successful ===
```



TIME COMPLEXITY

```

Void mergesort (int low, int high) {
    if (low < high) {
        int mid = (low + high) / 2;
        mergesort (low, mid);
        mergesort (mid + 1, high);
        merge (low, mid, high);
    }
}

```

$\rightarrow O(1)$
 $\rightarrow O(1)$
 $\rightarrow O(1)$
 $\rightarrow T(n/2)$
 $\rightarrow T(n/2)$
 $\rightarrow O(n)$

Recurrence Relation
 $T(n) = 2T(n/2) + O(n)$

By using master method $T(n) = aT(n/b) + f(n)$

$a=2, b=2,$

$\log_a b = \log_2 2 = 1$ $k=1, p=0$

Case - II $p > -1$

$T(n) = \Theta(n^k \log^{p+1} n)$

$T(n) = \Theta(n^1 \log^1 n)$

$T(n) = \Theta(n \log n)$

QUICK SORT

Step 1 : Start

Step 2 : Select an element as the pivot

Step 3 : partition the array :
 Elements smaller than the pivot $\rightarrow$ left side
 Elements greater than the pivot $\rightarrow$ Right side

Step 4 : place the pivot in its correct position

Step 5 : Apply Quick Sort to the left sub array.

Step 6 : Apply Quick Sort to the right sub array.

Step 7 : continue until each subarray contains zero or one element

Step 8 : Stop.

PROGRAM

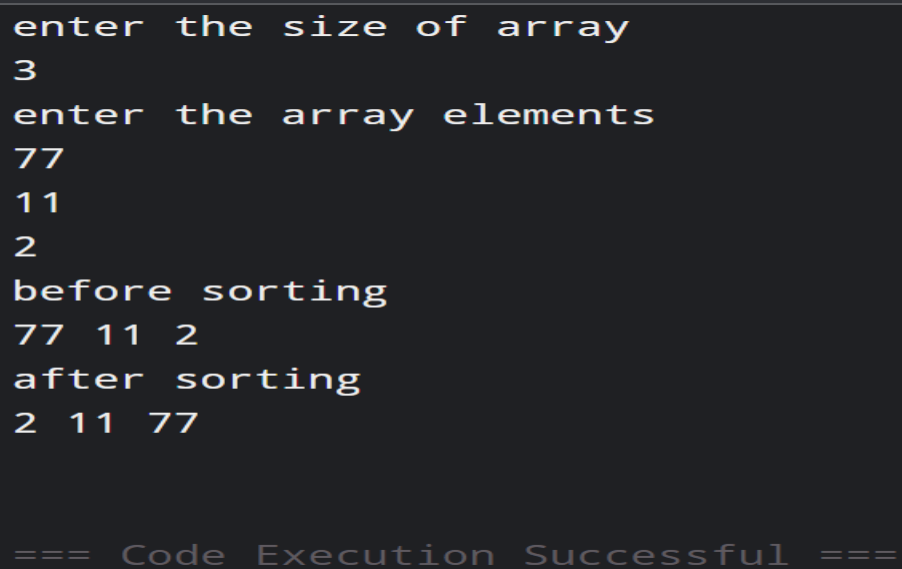
```
#include <iostream>
using namespace std;
class quicksort{
public:
    int size;
    int arr[100];
    int i;
    void arrayinput(){
        cout << "enter the size of array" << endl;
        cin >> size;
        cout << "enter the array elements" << endl;
        for (i = 0; i < size; i++){
            cin >> arr[i];
        }
    }
    void printarray(){
        for (i = 0; i < size; i++){
            cout << arr[i] << " ";
        }
        cout << endl;
    }
    int partition(int low, int high){
        int pivot = arr[high];
        int i = low - 1;
        int temp;
        for (int j = low; j < high; j++){
            if (arr[j] < pivot){
                i++;
                temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }

        temp = arr[i + 1];
        arr[i + 1] = arr[high];
        arr[high] = temp;
        return i + 1;
    }

    void quicklogic(int low, int high){
        if (low < high){
            int pi = partition(low, high);
            quicklogic(low, pi - 1);
            quicklogic(pi + 1, high);
        }
    }
}
```

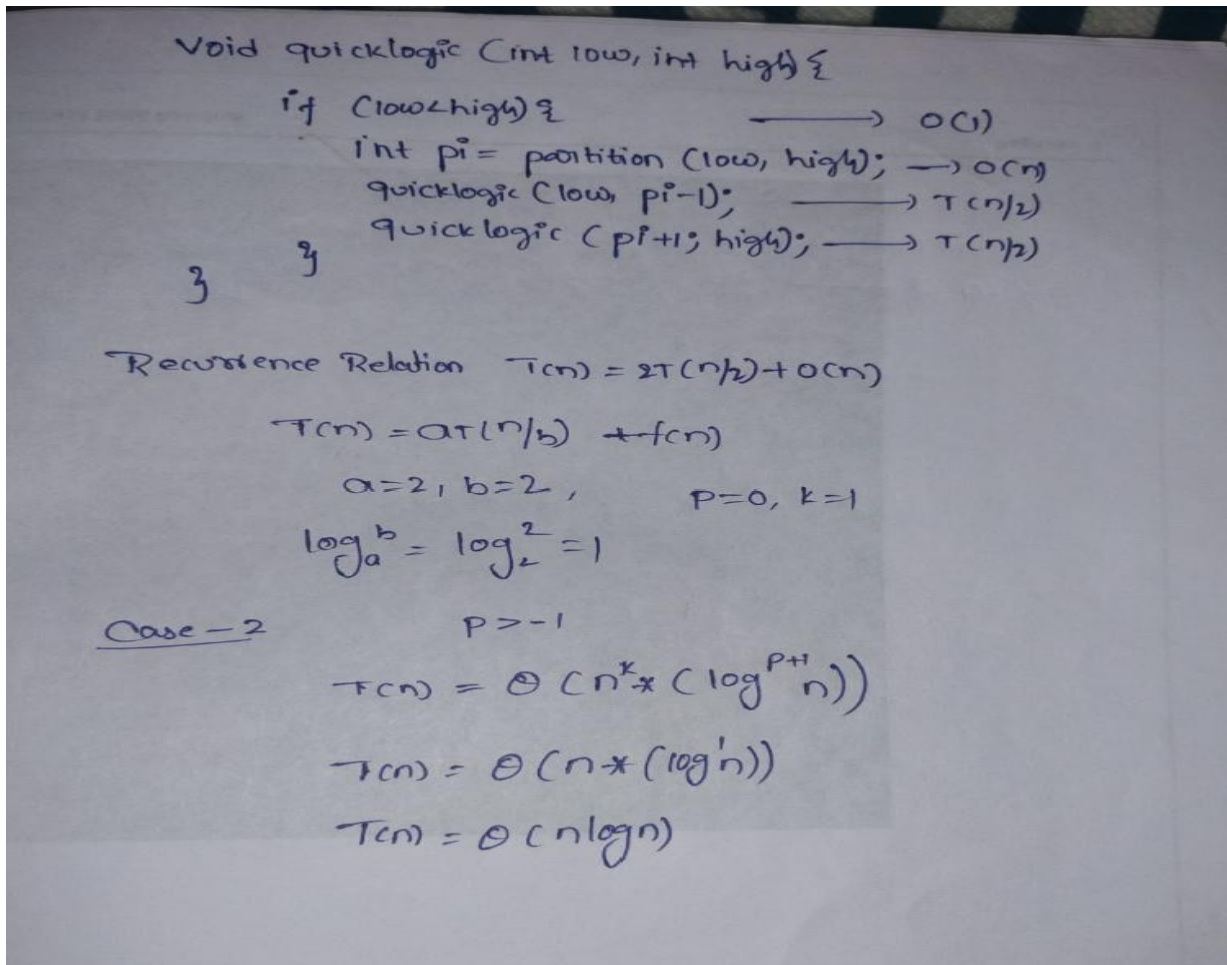
```
};  
  
int main()  
{  
    quicksort qs;  
    qs.arrayinput();  
    cout << "before sorting" << endl;  
    qs.printarray();  
    cout << "after sorting" << endl;  
    qs.quicklogic(0, qs.size - 1);  
    qs.printarray();  
    return 0;  
}
```

OUTPUT



```
enter the size of array  
3  
enter the array elements  
77  
11  
2  
before sorting  
77 11 2  
after sorting  
2 11 77  
  
=== Code Execution Successful ===
```

TIME COMPLEXITY



Conclusion:

I understood that different sorting algorithms use different ways to arrange data. Bubble, Selection and Insertion Sort are easy to understand and implement, but they can take more time for large data. Merge Sort and Quick Sort use a divide-and-conquer approach and are generally more efficient for large datasets. Overall, choosing an algorithm depends on the size and condition of the data and the required



Marwadi
University
Marwadi Chandarana Group



FACULTY OF TECHNOLOGY

Department of AI, ML & DS

01AI0506 | DAA Lab Manual